

Forecasting with Hyper-Trees

Authors Reply to Verification Report

1 Explanation

The differences between the reported and reproduced values have a well-understood source: the order in which a model’s compiled numerical code combines floating-point values is not fixed across machines. Every experiment is seeded, and re-running it on a fixed machine reproduces the numbers of the deterministic models identically. The differences therefore reflect the computing environment, not randomness or an error in the code. The tree, AR, and ARIMA fits vary across machines only through the accumulation order of floating-point operations. The deep-learning baselines (Deep-AR, TFT) and also our Hyper-TreeNet are subject to the same effect but additionally depend on non-deterministic GPU operations.

These computations are internal to each model fit: the tree construction in LightGBM, the matrix operations inside the neural network, and the maximum-likelihood optimisation inside the AR and ARIMA estimation. Because floating-point addition is not associative, a change in the order in which these values are combined can alter a result in its low-order digits. For the tree fits the order depends on both the compiled code path and the number of threads, so two machines can produce slightly different results even at an identical thread count. For the AR and ARIMA fits it is set by the compiled machine code, and for the neural network by the non-deterministic GPU operations already noted. Two installations can therefore agree exactly on some series and differ only slightly on others, which is the pattern the report shows for these baselines. Where the model is well conditioned the change is negligible. Where it is ill conditioned, as for the multiplicative seasonal recurrence on Tourism, it can be amplified. The effect also scales with the amount of data in each fit, so a per-series model can stay below the reported precision while a global model that pools many series may not.

A central point is that identical package versions do *not* guarantee an identical numerical environment. The same version number can sit on top of a different BLAS library and a different OpenMP runtime, which is exactly the gap between using `uv` and `conda`.

2 Evidence inside the reproduction report itself

Before turning to external sources, the verification report already contains the fingerprint of this mechanism.

Local models reproduce exactly, in the tables and in the figures. The local models in Tables 1 and 2 are fit on each series in isolation. Table 1 (Air Passengers) reproduces in full: every model, including the AR, AR-X, AutoARIMA, and AutoARIMA-X baselines, matches on every reported metric, with a single AR entry differing by 0.001 (MAE 44.292 versus 44.293), which the verifier records as reproduced with accuracy. In the multi-series local results (Table 2), every Hyper-Tree model, AutoETS, and every LightGBM variant again reproduces to the last digit, on all reported metrics and on every dataset. For these models the per-series training data is small enough for any accumulation-order effect to stay below the reported precision. Here the only rows that drift are the AR and ARIMA baselines and their exogenous-feature variants, through the platform-dependent estimation discussed in Section 3.3. These two baselines drift in different

ways, both consistent with the platform-dependent estimation rather than with any error. The fixed-order AR has no order to reselect, so its differences are the low-order arithmetic of the per-series fit. They are present on every dataset, including the single 0.001 cell on Air Passengers, and a little larger on the other series. AutoARIMA instead selects its order by an information criterion. On Air Passengers and on the five-series Australian Electricity set the selected orders are robust and it reproduces exactly, while on the datasets with many series a sub-rounding difference flips the selected order on a few of them, and those discrete changes aggregate into the drift. Air Passengers reproduces because it is a single clean series whose selected order is unambiguous, not because a single-series fit is intrinsically exact. The quantities derived from the Air Passengers Hyper-Tree-STL model also reproduce exactly: the estimated decomposition (Fig. 6), the time-varying parameters (Fig. 7), and the feature importances (Fig. 8). Because these figures expose the internal tree structure rather than an aggregate summary, their exact reproduction shows that the computation itself is deterministic, not merely that two summaries happen to coincide.

One global row reproduces fully, and several others in part. The same fingerprint is visible in the global table (Table 3), and this is important. One global row reproduces exactly on *every* error metric: the global LightGBM-AR model on Australian Electricity Demand matches on MAPE, sMAPE, WAPE, RMSE, MAE, and MASE. This is the smallest global dataset (five series), and even the pooled training data is small enough for the accumulation-order effect to remain below the reported precision. Several further rows reproduce exactly on a *subset* of metrics, and these include the Hyper-Tree models that are the subject of the paper: the global Hyper-Tree-ETS on Australian Electricity Demand matches exactly on MAPE, sMAPE, and MASE. The global Hyper-Tree-ETS on M3-Yearly and the global Hyper-Tree-AR on Australian Retail Turnover each match exactly on MASE. In every one of these rows the remaining differences are small relative to the reported values.

This pattern is the signature of floating-point accumulation rather than of a defect. Neither a coding error nor a stray source of randomness would reproduce an entire global row across two machines on every metric while leaving every other difference small. That is what the table shows: one row reproduces on all six metrics, the scale-free MASE coincides in several further Hyper-Tree rows, and the remaining differences are small relative to the reported values, somewhat larger only for the neural Hyper-TreeNet rows, which add a second computation on the GPU. A defect would not confine itself in this selective, metric-dependent way, so the structure of the differences is itself evidence for the explanation.

The Rossmann ablation preserves the conclusions it is built on. In the Rossmann ablation (Table 4) the differences are larger than in the global table, and they fall on all of the metrics. They remain small nevertheless. Only a handful of cells coincide at the printed precision, among them the MASE entries of the A2, A10, and A11 Hyper-TreeNet-AR variants and the A2 MAPE. What matters for the study is intact and every effect the ablation is designed to measure is preserved.

The embedding-dimension study reproduces exactly on the single series. Table G1 reports the Hyper-TreeNet-AR analysis at embedding dimensions $d \in \{1, 3, 5, 10\}$ for every dataset. Only the Air Passengers rows reproduce exactly. Because Air Passengers is a single, relatively short series, all four of its embedding dimensions match to the last digit on every error metric. On the multi-series datasets the values differ, in the same dataset-dependent way seen in the global table. That the neural Hyper-TreeNet-AR reproduces exactly on the single series at every embedding dimension, and only there, places the difference once more in the pooled multi-series computation rather than in the model or its embedding.

3 Corroboration in the literature and tool documentation

The behaviour we describe is well documented, both in the numerical-computing literature and in the official documentation of the exact libraries used in the paper.

3.1 Floating-point non-associativity in parallel reductions

That floating-point addition is non-associative, so that the order of a summation affects its result, is a textbook fact [1]. On parallel and multi-threaded hardware this directly implies non-reproducibility: the result of a reduction can vary from one parallel machine to another, and even from one run to another, with the effect worsening on heterogeneous hardware [2]. Demmel and Nguyen frame reproducibility as obtaining bitwise-identical results from repeated runs that may use different hardware resources, and show that ordinary parallel summation does not provide it without dedicated algorithms [3]. The OpenMP specification, which governs the threaded reductions inside LightGBM and the BLAS, does not fix the order in which partial results are combined, so bitwise determinism is not guaranteed [4]. Recent surveys document the same point for high-performance and scientific computing broadly [5], and the sensitivity of machine-learning training and inference pipelines to this effect is now explicitly recognised [4].

3.2 Gradient-boosted trees: LightGBM

The LightGBM documentation states the behaviour we rely on directly. The `deterministic` parameter is documented as producing stable results only for the same data and parameters *on a given machine*. The same note then states, verbatim, that “**in different systems, the results are expected to be different**” [7], listing different seeds, different library versions, binaries from different compilers, and different thread counts as cases in which the output is expected to change. A maintainer-confirmed issue describes a setup almost identical to ours, in which a model trained with `deterministic=True`, `force_row_wise=True`, and `num_threads=1` nonetheless produces different predictions on two different machines, with the maintainers pointing to the documentation note above [8]. Reducing the thread count to one eliminates the thread-scheduling source of variation but not the dependence on the compiled code path and the underlying numerical libraries, so single-threaded execution alone does not guarantee cross-machine agreement. A separate issue documents the exact compounding mechanism we describe: the first trees of a boosting run are bit-identical, a tiny divergence appears at one tree, and it accumulates into entirely different trees several hundred iterations later, with a visible effect on the final metrics [9]. The LightGBM method itself is described in [6].

3.3 The statistical estimators, BLAS, and the package manager

The AR and ARIMA estimators are compiled to machine code with numba, and each fit is an iterative maximum-likelihood optimisation whose objective is evaluated in that compiled code. The low-order digits of that objective therefore depend on the CPU and its instruction set. For AutoARIMA a second, discrete step amplifies this. The orders are selected by an information criterion, so a sub-rounding difference in the likelihood can move the chosen order on a few borderline series, which is what produces the small differences in those rows. The same estimators also call BLAS-backed linear algebra for their regression and exogenous-feature parts, and BLAS results themselves depend on the BLAS library and the platform. Intel introduced Conditional Numerical Reproducibility (CNR) in its Math Kernel Library precisely because the order of floating-point operations is affected by the number of threads, by threaded algorithms, and by run-time processor dispatch. Obtaining bitwise-reproducible results requires, among other conditions, a fixed numerical environment [10]. A user report on Intel’s own forum shows a LAPACK routine returning different results depending solely on the OpenMP thread count across two platforms, with no threading in the user’s own code [11].

The package manager matters for the same reason. By NumPy’s own account, the wheels distributed on PyPI (installed by `pip` and `uv`) bundle OpenBLAS, whereas a `conda` installation pulls in a separate BLAS package that may be OpenBLAS or MKL depending on the channel [12]. Consequently, two environments reporting identical NumPy version numbers can execute on different linear-algebra libraries, which is exactly the difference between the authors’ `uv` environment and the verifier’s `conda` environment.

3.4 Neural networks and the deep-learning baselines: PyTorch

The neural-network component and the deep-learning baselines run on PyTorch, whose documentation states that completely reproducible results are not guaranteed across releases, commits, or different platforms, and that results need not be reproducible between CPU and GPU execution even with identical seeds [13]. Named sources of this nondeterminism include cuDNN algorithm selection and non-deterministic atomic additions on the GPU. A tracked issue shows a small model that is reproducible on a single machine yet yields different results on different hardware, even with deterministic settings enabled [14]. This supports both the residual drift in the Hyper-TreeNet component and the treatment of the stochastic deep baselines as reproducible only up to sampling and hardware variance.

4 Conclusion

The differences reported in the Verification Report are an expected and documented property of floating-point computation in compiled numerical code. The reproduction report itself confirms determinism for the per-series local fits, namely every Hyper-Tree model, AutoETS, and LightGBM variant, together with the figures derived from the tree structure, and shows the characteristic floating-point signature in the global table, where one row matches on all six metrics, several Hyper-Tree rows match on a subset of metrics, and the remaining differences are small relative to the reported values. The only local exceptions, the AR and ARIMA estimators, drift through the same platform-dependent estimation. The supporting references show that this is the established behaviour of the specific libraries used, namely LightGBM, BLAS/MKL, and PyTorch, and of parallel floating-point reductions in general. No ranking or conclusion in the paper is affected.

References

- [1] D. Goldberg, “What every computer scientist should know about floating-point arithmetic,” *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.
- [2] S. Collange, D. Defour, S. Graillat, and R. Iakymchuk, “Numerical reproducibility for the parallel reduction on multi- and many-core architectures,” *Parallel Computing*, vol. 49, pp. 83–97, 2015.
- [3] J. Demmel and H. D. Nguyen, “Parallel reproducible summation,” *IEEE Transactions on Computers*, vol. 64, no. 7, pp. 2060–2070, 2015.
- [4] S. Shanmugavelu, M. Taillefumier, C. Culver, O. R. Hernandez, M. Coletti, and A. Sedova, “Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications,” in *SC’24 Workshops*, IEEE/ACM, 2024.
- [5] B. A. Antunes and D. R. C. Hill, “Reproducibility, replicability, and repeatability: A survey of reproducible research with a focus on high performance computing,” *Computer Science Review*, vol. 53, art. 100655, 2024.
- [6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [7] LightGBM, “Parameters,” LightGBM documentation. Accessed June 2026.
<https://lightgbm.readthedocs.io/en/latest/Parameters.html>
- [8] “Deterministic and related flags don’t guarantee same result on different machines,” LightGBM GitHub issue #6683, 2024.
<https://github.com/microsoft/LightGBM/issues/6683>
- [9] “Models not reproducible on cpu,” LightGBM GitHub issue #2598, 2019.
<https://github.com/microsoft/LightGBM/issues/2598>
- [10] Intel, “Obtaining numerically reproducible results / Conditional Numerical Reproducibility (CNR),” Intel oneMKL Developer Guide and Reference. Accessed June 2026.
<https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/current/conditional-numerical-reproducibility-control.html>
- [11] “MKL library produces different results depending on OpenMP thread count,” Intel Community forum, 2024.
<https://community.intel.com/t5/Intel-oneAPI-Math-Kernel-Library/MKL-library-produces-different-results-depending-on-OpenMP/m-p/1559629>
- [12] NumPy project, “Building from source” (BLAS and LAPACK selection, “OpenBLAS is the NumPy default”, the PyPI wheels bundle OpenBLAS, and the conda defaults channel builds against MKL), NumPy documentation. Accessed June 2026.
<https://numpy.org/doc/stable/building/>
- [13] PyTorch, “Reproducibility,” PyTorch documentation. Accessed June 2026.
<https://docs.pytorch.org/docs/stable/notes/randomness.html>
- [14] “Model not reproducible on different hardware, but same platform (Docker),” PyTorch GitHub issue #38219, 2020.
<https://github.com/pytorch/pytorch/issues/38219>